UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE TECNOLOGIA
DEPARTAMENTO DE ENGENHARIA DE COMPUTAÇÃO E AUTOMAÇÃO
CURSO DE ENGENHARIA DE COMPUTAÇÃO

# Relatório da Tarefa 06 - Sincronização e Gerenciamento de Escopo de Variáveis em OpenMP

**DCA3703 - PROGRAMAÇÃO PARALELA - T01 (2025.2)**

WERBERT ARLES DE SOUZA BARRADAS

20250070655

Docente: Professor Doutor SAMUEL XAVIER DE SOUZA

Natal, 2 de setembro de 2025

# Lista de Figuras

# Sumário


**Lista de Figuras** . . . . . 2

**Sumário** . . . . . 3

**1 INTRODUÇÃO** . . . . . 4

**2 METODOLOGIA DO EXPERIMENTO** . . . . . 5

**2.1 Estrutura e Geração da Lista Encadeada** . . . . . 5

**2.2 Implementação do Processamento Paralelo** . . . . . 5

**2.3 Análise das Diretivas** . . . . . 6

**2.4 Diretivas não utilizadas** . . . . . 6

2.4.1 main com região Paralela Otimizada (single e task) . . . . . 7

**3 ANÁLISE E DISCUSSÃO** . . . . . 8

**3.1 Processamento dos Nós** . . . . . 8

3.1.1 Todos os nós foram processados? . . . . . 8

3.1.2 Algum nó foi processado mais de uma vez ou ignorado? . . . . . 8

**3.2 Comportamento e Sincronização** . . . . . 8

3.2.1 O comportamento muda entre execuções? . . . . . 8

3.2.2 Como garantir que cada nó seja processado uma única vez e por apenas uma tarefa? . . . . . 9

**3.3 Conclusão** . . . . . 10

**Anexo A: Código Lista**$_{encadeada.c}$ . . . . . 11

# 1 Introdução

O processamento de estruturas de dados dinâmicas, como listas encadeadas, em ambientes de programação paralela apresenta desafios únicos. Diferente de laços iterativos com um número fixo de repetições, onde a carga de trabalho é distribuída de maneira previsível, o processamento de nós de uma lista exige um modelo de paralelismo mais flexível. O **modelo de tarefas** da API OpenMP é ideal para essa finalidade, permitindo a criação e o agendamento dinâmico de unidades de trabalho.

O objetivo deste projeto é explorar e demonstrar a eficácia do modelo de tarefas do OpenMP para processar uma lista encadeada. Para isso, foi desenvolvido um programa que cria uma lista de arquivos fictícios e, em uma região paralela, cria uma tarefa para processar cada nó da lista.

O projeto analisa o comportamento da implementação e busca responder a questões fundamentais sobre o paralelismo:

1. **Processamento Completo:** A abordagem com tarefas garante que todos os nós da lista são processados, sem ignorar nenhum.
2. **Unicidade do Processamento:** O modelo evita que nós sejam processados mais de uma vez, um risco comum em ambientes concorrentes.
3. **Determinismo na Execução:** A análise da saída do programa revela se a ordem de execução das tarefas é consistente ou se muda a cada execução.
4. **Garantia de Correção:** O estudo busca identificar a forma correta de implementar o paralelismo para garantir que cada nó seja processado de forma única por uma única tarefa.

Através da implementação e da análise do programa, o relatório visa ilustrar os ganhos de flexibilidade e a importância de gerenciar corretamente o acesso a recursos em um modelo de paralelismo baseado em tarefas.

# 2 Metodologia do Experimento

O cerne deste projeto consiste na criação de uma lista encadeada e no desenvolvimento de uma abordagem de programação paralela que garanta o processamento de cada um de seus nós de forma correta e eficiente. Para isso, a implementação utiliza o modelo de tarefas da API OpenMP, que se mostra ideal para estruturas de dados dinâmicas onde o número de elementos não é fixo.

## 2.1 Estrutura e Geração da Lista Encadeada

A lista encadeada foi implementada com a estrutura `Node`, contendo um campo para o nome do arquivo fictício e um ponteiro para o próximo nó. Funções de suporte foram desenvolvidas para gerenciar a lista, incluindo a criação de novos nós (`createNode`) e a inserção no final da lista (`insertNode`).

```
typedef struct Node {
    char filename[50];
    struct Node* next;
} Node;
```

## 2.2 Implementação do Processamento Paralelo

A estratégia de paralelização empregada garante que cada nó da lista seja processado por uma tarefa independente, aproveitando os múltiplos núcleos de processamento. A lógica de controle é encapsulada em uma região paralela, onde a criação das tarefas é gerenciada de forma segura.

1. **Criação da Região Paralela:** A diretiva `#pragma omp parallel` inicia o ambiente de execução concorrente, criando um grupo de *threads* prontas para a execução das tarefas.

2. **Serialização da Criação de Tarefas:** A diretiva `#pragma omp single` é utilizada para garantir que apenas uma *thread* da equipe seja responsável por percorrer a lista encadeada e criar as tarefas. Isso é crucial para evitar condições de corrida na travessia da lista e garantir a unicidade do processamento de cada nó.

3. **Criação das Tarefas:** Dentro do laço de iteração da lista, a diretiva `#pragma omp task` é aplicada para cada nó. Isso agrupa a chamada à função `processNode` em

uma tarefa que é agendada para ser executada por qualquer *thread* disponível no *pool*.

## 2.3 Análise das Diretivas

O programa usou o modelo de tarefas para processar a lista encadeada. A combinação de parallel, single e task é o que garante a execução correta e eficiente.

- **#pragma omp parallel:** Essa diretiva cria a equipe de threads que irá trabalhar na região paralela. É o ponto de entrada para o processamento concorrente.
- **#pragma omp single:** Esta é a diretiva crucial para o seu programa. Ela garante que o bloco de código que a segue seja executado por apenas um thread da equipe. Isso foi essencial para percorrer a lista encadeada de forma segura e criar uma tarefa para cada nó, evitando que múltiplos threads tentassem fazer o mesmo, o que causaria erros.
- **#pragma omp task:** A diretiva omp task é a base da sua solução. Ela define um bloco de código como uma tarefa independente. Cada tarefa é agendada em um "pool"e pode ser executada por qualquer thread que esteja disponível. Esse modelo é ideal para listas encadeadas, pois o trabalho é distribuído dinamicamente entre os threads, sem a necessidade de um laço pré-determinado como o for.

A combinação de **#pragma omp single** e a barreira implícita no final da região paralela garante a unicidade do processamento e a integridade da memória, prevenindo a liberação da lista antes que todas as tarefas sejam concluídas."

## 2.4 Diretivas não utilizadas

As seguintes diretivas e cláusulas não foram necessárias para a solução da Tarefa 7, mas são importantes em outros cenários de programação paralela:

- **#pragma omp master:** Esta diretiva executa um bloco de código apenas pelo thread mestre da equipe. A diretiva single foi usada no seu código por ser mais flexível, permitindo que qualquer thread disponível inicie a criação das tarefas.
- **#pragma omp barrier:** Utilizada para sincronizar explicitamente todos os threads. Seu programa não precisou dela, pois o final da região parallel já possui uma barreira implícita, garantindo que todas as tarefas sejam concluídas antes do programa prosseguir.

- **#pragma omp taskwait:** Faz com que a thread que a encontra espere por todas as tarefas que ela mesma criou. No seu caso, a barreira implícita no final da região parallel já cumpre essa função, garantindo que o programa não termine antes que todas as tarefas sejam executadas.

### 2.4.1 main com região Paralela Otimizada (single e task)

```
int main() {
Node* head = NULL;
int num_files = 15;
char filename_buffer[50];
// Cria a lista encadeada com nomes de arquivos fictcios
for (int i = 0; i < num_files; i++) {
    snprintf(filename_buffer, sizeof(filename_buffer), "arquivo_%d.txt"
        , i + 1);
    insertNode(&head, filename_buffer);
}
// Regiao paralela para percorrer e processar a lista com tarefas
#pragma omp parallel
{
    #pragma omp single
    {
        printf("As tarefas foram processadas pela thread: %d\n",
            omp_get_thread_num());
        Node* current = head;
        while (current != NULL) {
            #pragma omp task
            {
                processNode(current);
            }
            current = current->next;
        }
    }
}
// Libera a memoria alocada para a lista
freeList(&head);

return 0;
}
```

Listing 2.1 – Versao Paralela com single e task

# 3 Análise e Discussão

Após a execução do programa de processamento de lista encadeada, a análise do comportamento da aplicação em um ambiente de programação paralela levanta questionamentos fundamentais que são respondidos a seguir.

## 3.1 Processamento dos Nós

### 3.1.1 Todos os nós foram processados?

Sim, todos os nós da lista encadeada foram processados. A combinação do laço de iteração `while` com a diretiva `#pragma omp single` garante que a lista seja percorrida completamente por uma única *thread* da equipe. A cada iteração, uma tarefa (`#pragma omp task`) é criada para processar o nó atual. O sistema de execução do OpenMP é responsável por agendar e executar todas as tarefas criadas até que o trabalho seja finalizado.

### 3.1.2 Algum nó foi processado mais de uma vez ou ignorado?

Não, nenhum nó foi processado mais de uma vez, tampouco ignorado. O uso da diretiva `#pragma omp single` é a chave para a garantia da unicidade do processamento. Ao restringir a criação de tarefas a uma única *thread*, evita-se a condição de corrida que poderia ocorrer se múltiplas *threads* tentassem percorrer a lista simultaneamente. Isso garante que cada nó seja agendado para processamento exatamente uma vez.

## 3.2 Comportamento e Sincronização

### 3.2.1 O comportamento muda entre execuções?

Sim, o comportamento de execução muda a cada vez que o programa é executado. A ordem em que os arquivos são processados (e impressos no console) é não determinística. Isso acontece porque a execução das tarefas não segue uma ordem fixa. As tarefas são distribuídas para as *threads* que se tornam disponíveis, e o agendamento pode variar dependendo da carga do sistema e do escalonador de *threads*.

### 3.2.2 Como garantir que cada nó seja processado uma única vez e por apenas uma tarefa?

A garantia de que cada nó seja processado de forma única é alcançada pela combinação estratégica de diretivas OpenMP. A diretiva `#pragma omp single` serializa a fase de criação de tarefas, assegurando uma iteração segura pela lista. Embora a criação seja sequencial, a execução das tarefas é paralela, otimizando o desempenho. A barreira implícita no final da região `parallel` atua como um mecanismo de sincronização final, garantindo que todas as tarefas sejam concluídas antes que o programa prossiga.

## 3.3 Conclusão

A partir da análise e dos resultados observados, conclui-se que o experimento demonstrou com sucesso a aplicação do modelo de tarefas do OpenMP para o processamento de uma estrutura de dados dinâmica, como a lista encadeada.

A implementação validou a importância da sincronização correta em ambientes de programação concorrente. O uso estratégico da diretiva **#pragma omp single** assegurou que, mesmo dentro de uma região paralela, a criação das tarefas fosse realizada de forma segura e serial por apenas um thread. Isso garantiu a unicidade do processamento, provando que todos os nós foram visitados e processados uma única vez, sem a ocorrência de condições de corrida ou a perda de dados.

Adicionalmente, a observação do comportamento de execução reforçou um conceito fundamental do paralelismo: a não-determinismo. A variação na ordem de processamento dos nós entre as execuções do programa é uma característica natural do modelo de tarefas e ilustra como o OpenMP distribui dinamicamente a carga de trabalho entre os threads disponíveis.

Em suma, a Tarefa 7 serviu como uma demonstração prática e robusta da flexibilidade e do poder do modelo de tarefas, evidenciando que o OpenMP é uma ferramenta eficaz para o processamento de estruturas complexas, desde que as diretivas de sincronização e de controle de fluxo sejam aplicadas corretamente para garantir a exatidão e a segurança da aplicação.

**list_task.c**

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <omp.h>

// Definição da estrutura do nó da lista encadeada
typedef struct Node {
    char filename[50];
    struct Node* next;
} Node;

// Função para criar um novo nó
Node* createNode(const char* filename) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (newNode == NULL) {
        perror("Erro ao alocar memória para o nó");
        exit(1);
    }
    strncpy(newNode->filename, filename, sizeof(newNode->filename) - 1);
    newNode->filename[sizeof(newNode->filename) - 1] = '\0';
    newNode->next = NULL;
    return newNode;
}

// Função para inserir um nó no final da lista
void insertNode(Node** head, const char* filename) {
    Node* newNode = createNode(filename);
    if (*head == NULL) {
        *head = newNode;
    } else {
        Node* temp = *head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}

// Função para processar o nó
void processNode(Node* node) {
    if (node) {
        printf("Processando arquivo: %s, pela thread: %d\n", node->filename, omp_get_thread_num());
    }
}

// Função para liberar a memória da lista
void freeList(Node** head) {
    Node* current = *head;
    Node* next;
```

```c
    while (current != NULL) {
        next = current->next;
        free(current);
        current = next;
    }
    *head = NULL;
}

int main() {
    Node* head = NULL;
    int num_files = 15;
    char filename_buffer[50];

    // Cria a lista encadeada com nomes de arquivos fictícios
    for (int i = 0; i < num_files; i++) {
        snprintf(filename_buffer, sizeof(filename_buffer), "arquivo_%d.txt", i + 1);
        insertNode(&head, filename_buffer);
    }

    // Região paralela para percorrer e processar a lista com tarefas
    #pragma omp parallel
    {
        #pragma omp single
        {
            printf("As tarefas foram processadas pela thread: %d\n", omp_get_thread_num());
            Node* current = head;
            while (current != NULL) {
                #pragma omp task
                {
                    processNode(current);
                }
                current = current->next;
            }
        }
    }

    // Libera a memória alocada para a lista
    freeList(&head);

    return 0;
}
```